

1. What is the time complexity of the following loop?

```
for (int i = 0; i < n; i = i + 5)
{
    // Constant time operation
}
```

- A) $O(5n)$
- B) $O(n)$
- C) $O(n/5)$
- D) $O(1)$

2. Analyze the complexity of these sequential loops:

```
for (int i = 0; i < n; i++)
{ /* code */ }
for (int j = 0; j < m; j++)
{ /* code */ }
```

- A) $O(n \times m)$
- B) $O(n^2)$
- C) $O(n + m)$
- D) $O(\max(n, m))$

3. What is the worst-case complexity of this nested loop?

```
for (int i = 0; i < n; i++) {
    for (int j = 1; j < n; j = j * 2) {
        // Constant time operation
    }
}
```

- A) $O(n^2)$
- B) $O(n \log n)$
- C) $O(n)$
- D) $O(\log n)$

4. Consider the following conditional. What is the worst-case complexity?

```
if (condition) {
    for (int i = 0; i < n; i++) { /* linear work */ }
} else {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) { /* quadratic work */ }
    }
}
```

- A) $O(n)$
- B) $O(n^2)$
- C) $O(n + n^2)$
- D) $O(n^3)$

5. What is the complexity when the inner loop depends on the outer loop index as follows?

```
for (int i = 0; i < n; i++) {
```

```

    for (int j = 0; j < i; j++) {
        // Constant time operation
    }
}

```

- A) $O(n)$
- B) $O(n \log n)$
- C) $O(n^2)$
- D) $O(i \times n)$

6. Find the complexity of the following loop:

```

for (int i = n; i > 1; i = i / 2) {
    // Constant time operation
}

```

- A) $O(n)$
- B) $O(1)$
- C) $O(\log n)$
- D) $O(n/2)$

7. What is the complexity of a loop that runs from 1 to n^2 ?

```

for (int i = 0; i < n * n; i++) {
    // Constant time operation
}

```

- A) $O(n)$
- B) $O(n^2)$
- C) $O(2n)$
- D) $O(\sqrt{n})$

8. If a loop calls a function `foo(n)` which has a complexity of $O(n)$, what is the total complexity?

```

for (int i = 0; i < n; i++) {
    foo(n);
}

```

- A) $O(n)$
- B) $O(n^2)$
- C) $O(2n)$
- D) $O(n!)$

9. What is the complexity of the following code?

```

int i = 1;
while (i * i <= n) {
    i++;
}

```

- A) $O(n)$
- B) $O(\log n)$

- C) $O(\sqrt{n})$
- D) $O(n^2)$

10. Analyze the following triple nested loop:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < 100; j++) {  
        for (int k = 0; k < n; k++) {  
            // Constant time  
        }  
    }  
}
```

- A) $O(100n^2)$
- B) $O(n^3)$
- C) $O(n^2)$
- D) $O(n)$

11. In a worst-case analysis, if an if-else statement has branches with $O(1)$ and $O(\log n)$ respectively, which is chosen?

- A) $O(1)$
- B) $O(\log n)$
- C) $O(1 + \log n)$
- D) $O(n)$

12. What is the complexity of this loop?

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j = j + i) {  
        // Constant time  
    }  
}
```

- A) $O(n^2)$
- B) $O(n \log n)$
- C) $O(n)$
- D) $O(\log n)$

13. What happens to the complexity if a loop variable is multiplied by a constant $k > 1$ in each iteration?

- A) It becomes linear
- B) It becomes quadratic
- C) It becomes logarithmic
- D) It becomes exponential

14. What is the complexity of the following code?

```
for (int i = 0; i < n; i++) {  
    i++; // incrementing i again inside  
}
```

- A) $O(n)$
- B) $O(n/2)$
- C) $O(1)$
- D) $O(n^2)$

15. If a condition checks if (`array.isSorted()`) before performing a $O(1)$ task, where `isSorted()` is $O(n)$, what is the worst-case complexity?

- A) $O(1)$
- B) $O(n)$
- C) $O(n^2)$
- D) $O(\log n)$

16. Analyze the following loop:

```
for (int i = 1; i < n; i++) {
    for (int j = n; j > 0; j = j - 2) {
        // Constant time
    }
}
```

- A) $O(n)$
- B) $O(n^2)$
- C) $O(n \log n)$
- D) $O(n/2)$

17. What is the complexity of the following loop structure?

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < 5; j++) {
        // Constant time
    }
}
```

- A) $O(5n)$
- B) $O(n^2)$
- C) $O(n)$
- D) $O(5)$

18. What is the complexity of this while loop?

```
int p = 0;
for (int i = 1; p <= n; i++) {
    p = p + i;
}
```

- A) $O(n)$
- B) $O(\log n)$
- C) $O(\sqrt{n})$
- D) $O(n^2)$

19. What is the complexity of a loop that iterates 2^k times where $k = \log_2 n$?

- A) $O(2^n)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(k)$

20. In an if (A && B) statement, if A takes $O(n)$ and B takes $O(n^2)$, what is the worst-case time complexity of the evaluation?

- A) $O(n)$
- B) $O(n^2)$
- C) $O(n^3)$
- D) $O(n+n^2)$

Answer Explanations

- * B: Even though i increments by 5, the number of iterations is $n/5$. In Big O, constants like $1/5$ are dropped, so it is $O(n)$.
- * C: Sequential loops are additive ($n + m$). While $O(\max(n, m))$ is also technically correct, $O(n + m)$ is the standard representation for two different input variables.
- * B: The outer loop runs n times. The inner loop variable j doubles each time, which is $\log n$ iterations. $n \times \log n = O(n \log n)$.
- * B: Worst-case analysis assumes the path that takes the longest time. The else branch is $O(n^2)$, which dominates the $O(n)$ branch.
- * C: This forms a triangular sum: $1 + 2 + 3 + \dots + n$. The sum is $\frac{n(n-1)}{2}$, which simplifies to $O(n^2)$.
- * C: Dividing the loop variable by 2 in each step is the inverse of doubling; it takes $\log_2 n$ steps to reach 1.
- * B: The loop runs $n \times n$ times, which is directly $O(n^2)$.
- * B: You are doing n units of work (the function) n times (the loop). $n \times n = O(n^2)$.
- * C: The condition $i^2 \leq n$ is equivalent to $i \leq \sqrt{n}$. The loop runs $\sqrt{n}$ times.
- * C: The middle loop runs exactly 100 times, which is a constant. $n \times 100 \times n = 100n^2$. Drop the constant to get $O(n^2)$.
- * B: Big O focuses on the upper bound. Between constant and logarithmic, the logarithmic branch is the "worst" case.
- * B: This is the Harmonic Series: $n/1 + n/2 + n/3 + \dots + n/n$. This sum is $n(1 + 1/2 + 1/3 + \dots + 1/n)$, which is $n \cdot \ln(n)$, or $O(n \log n)$.
- * C: Any time a loop variable grows geometrically (multiplied by k), the number of iterations is logarithmic relative to the limit.
- * A: Incrementing i twice per loop effectively makes the loop run $n/2$ times. Asymptotically, this is still $O(n)$.
- * B: To evaluate the condition, the computer must run the `isSorted()` function. Since that function takes $O(n)$, the overall complexity is $O(n)$ even if the code inside the if is $O(1)$.
- * B: The outer loop is n . The inner loop is $n/2$ (decrementing by 2). $n \times (n/2) = n^2/2$, which is $O(n^2)$.
- * C: The inner loop is constant (always 5 times). Therefore, it's just $5 \times n$, which is $O(n)$.
- * C: p grows as the sum of integers: 1, 3, 6, 10 $\dots$ (triangular numbers). The k -th triangular number is approximately $k^2/2$. Setting $k^2/2 \approx n$ gives $k \approx \sqrt{2n}$, which is $O(\sqrt{n})$.
- * C: $2^{\log_2 n}$ is mathematically equal to n . Therefore, the loop runs n times.
- * B: In the worst case (if A is true), the computer must evaluate both. The sum $n + n^2$ is dominated by the n^2 term.

1. What does the 'Big O' notation specifically represent in algorithm analysis?
 - A) The tight bound of the execution time
 - B) The upper bound of the execution time
 - C) The lower bound of the execution time
 - D) The average execution time

2. What is the worst-case time complexity of searching for an element in an unsorted array of size n ?
 - A) $O(1)$
 - B) $O(\log n)$
 - C) $O(n)$
 - D) $O(n \log n)$

3. Which of the following growth rates is the slowest (most efficient)?
 - A) $O(n^2)$
 - B) $O(n \log n)$
 - C) $O(n)$
 - D) $O(\log n)$

4. What is the worst-case time complexity of the following code snippet?

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        // constant time operation  
    }  
}
```

 - A) $O(n)$
 - B) $O(n^2)$
 - C) $O(2n)$
 - D) $O(n^3)$

5. Binary search has a worst-case time complexity of $O(\log n)$. What condition must be met for this to be true?
 - A) The array must be empty
 - B) The array must be sorted
 - C) The array must contain only integers
 - D) The array size must be a power of 2

6. What is the worst-case complexity of Quicksort?
 - A) $O(n \log n)$
 - B) $O(n^2)$
 - C) $O(n)$
 - D) $O(\log n)$

7. Which complexity represents an algorithm where the execution time doubles with each additional element in the input?
 - A) $O(n^2)$
 - B) $O(2^n)$
 - C) $O(n!)$
 - D) $O(\log n)$

8. In Big O notation, how is the expression $3n^2 + 10n + 5$ simplified?

- A) $O(3n^2)$
- B) $O(n^2)$
- C) $O(n^2 + n)$
- D) $O(n^3)$

9. What is the worst-case time complexity of Merge Sort?

- A) $O(n \log n)$
- B) $O(n^2)$
- C) $O(n)$
- D) $O(1)$

10. Analyze the following loop:

```
for (int i = 1; i < n; i = i * 2) {  
    // constant time operation  
}
```

- A) $O(n)$
- B) $O(\log n)$
- C) $O(n \log n)$
- D) $O(1)$

11. If an algorithm's complexity is $O(1)$, how does the execution time change as the input size increases?

- A) It increases linearly
- B) It increases quadratically
- C) It remains constant
- D) It increases logarithmically

12. What is the worst-case time complexity for inserting an element at the beginning of an array of size n ?

- A) $O(1)$
- B) $O(n)$
- C) $O(\log n)$
- D) $O(n^2)$

13. Which analysis case is most useful for providing a 'guaranteed' performance limit?

- A) Best Case
- B) Average Case
- C) Worst Case
- D) Amortized Case

14. What is the worst-case time complexity of finding the maximum element in a Max-Heap of size n ?

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n \log n)$

15. According to the Master Theorem, what is the complexity of an algorithm that divides a problem into two subproblems of size $n/2$ and combines them in $O(n)$ time?

- A) $O(n)$
- B) $O(n \log n)$
- C) $O(n^2)$
- D) $O(\log n)$

16. What is the worst-case time complexity of Breadth-First Search (BFS) on a graph with V vertices and E edges?

- A) $O(V)$
- B) $O(E)$
- C) $O(V + E)$
- D) $O(V \times E)$

17. An algorithm has a complexity of $O(n!)$. What happens to the time taken if n increases from 10 to 11?

- A) It increases by 1 unit
- B) It doubles
- C) It increases by a factor of 11
- D) It increases by 11 units

18. What is the worst-case time complexity of Bubble Sort?

- A) $O(n)$
- B) $O(n \log n)$
- C) $O(n^2)$
- D) $O(1)$

19. What is the worst-case complexity of accessing a specific index in a Singly Linked List of size n ?

- A) $O(1)$
- B) $O(n)$
- C) $O(\log n)$
- D) $O(n^2)$

20. If an algorithm runs in $O(\sqrt{n})$ time, how does it compare to $O(\log n)$ as n becomes very large?

- A) $O(\sqrt{n})$ is faster
- B) $O(\log n)$ is faster
- C) They are equivalent
- D) It depends on the constants

21. What is the worst-case time complexity of the Traveling Salesperson Problem (TSP) using a brute-force approach?

- A) $O(n^2)$
- B) $O(2^n)$
- C) $O(n!)$
- D) $O(n^n)$

22. What is the worst-case time complexity for searching in a Binary Search

Tree (BST)?

- A) $O(\log n)$
- B) $O(n)$
- C) $O(1)$
- D) $O(n \log n)$

23. Analyze the complexity of this recursive function:

```
int func(int n) {  
    if (n <= 1) return 1;  
    return func(n - 1) + func(n - 1);  
}
```

- A) $O(n)$
- B) $O(n^2)$
- C) $O(2^n)$
- D) $O(\log n)$

24. What is the formal definition of $T(n) = O(g(n))$?

- A) $T(n) \leq c \cdot g(n)$ for all $n \geq n_0$
- B) $T(n) \geq c \cdot g(n)$ for all $n \geq n_0$
- C) $T(n) = g(n)$
- D) $T(n) < g(n)$

25. What is the worst-case time complexity of building a heap from an unordered array using the 'bottom-up' method?

- A) $O(n \log n)$
- B) $O(n)$
- C) $O(n^2)$
- D) $O(\log n)$

Answer Key

Q#	Ans	Q#	Ans	Q#	Ans	Q#	Ans	Q#	Ans
1	B	6	B	11	C	16	C	21	C
2	C	7	B	12	B	17	C	22	B
3	D	8	B	13	C	18	C	23	C
4	B	9	A	14	A	19	B	24	A
5	B	10	B	15	B	20	B	25	B

Here is the comprehensive explanation for each question to help you understand the logic behind the worst-case time complexity analysis.

Answer Explanations

* B: Big O (O) provides an asymptotic upper bound. It guarantees that the algorithm will never take longer than this bound for large enough input sizes.

* C: In an unsorted array, the element you are looking for could be at the very last position or not there at all, forcing you to check all n elements.

* D: As n grows, $\log n$ increases much more slowly than linear (n), linear-logarithmic ($n \log n$), or quadratic (n^2) functions.

* B: For every 1 iteration of the outer loop (n total), the inner loop runs n times. Total operations: $n \times n = n^2$.

* B: Binary Search works by comparing the middle element and discarding half of the search space. This logic only works if the data is ordered (sorted).

* B: Quicksort's performance depends on the pivot. If the pivot is always the smallest or largest element (common in sorted arrays), the partitions are highly unbalanced, leading to $O(n^2)$.

* B: An exponential growth 2^n means that for every +1 added to n , the total operations are multiplied by 2 ($2^{n+1} = 2 \cdot 2^n$).

* B: In Big O analysis, we drop constants (3, 10, 5) and keep only the dominant term (n^2), as lower-order terms become insignificant at infinity.

* A: Merge Sort always divides the array exactly in half and takes $O(n)$ time to merge. This consistency guarantees $O(n \log n)$ regardless of the input order.

* B: Because the loop variable i doubles ($i = i \times 2$) rather than increments, the number of steps required to reach n is $\log_2 n$.

* C: $O(1)$ is "constant time." It means the complexity does not depend on the input size; it takes the same time for 10 elements as it does for 10 million.

* B: Arrays are stored in contiguous memory. To put something at index 0, you must shift all n existing elements one slot to the right.

* C: Engineers use the Worst Case to define a "safety margin." It ensures the system can handle the heaviest possible load without failing.

* A: A Max-Heap is organized so the largest value is always at the root (the first element), allowing for immediate access.

* B: This is the standard recurrence for Divide and Conquer algorithms like Merge Sort ($T(n) = 2T(n/2) + O(n)$), which solves to $O(n \log n)$ via the Master Theorem.

* C: BFS must visit every vertex (V) and, in the worst case, check every edge (E) connected to those vertices to see if they lead to unvisited nodes.

* C: Factorial growth is defined as $n! = n \times (n-1) \times \dots \times 1$. Therefore, $11!$ is exactly $11 \times 10!$.

* C: Even with minor optimizations, the worst-case for Bubble Sort (a reverse-sorted list) requires every element to be compared against every other element in nested loops.

* B: Unlike arrays, Linked Lists do not have indexes in memory. You must start at the head and follow the "next" pointers n times to reach the end.

* B: While $\sqrt{n}$ grows slowly, it is still a polynomial ($n^{0.5}$). Logarithmic functions ($O(\log n)$) always grow slower than any positive polynomial power as n approaches infinity.

* C: In the Traveling Salesperson Problem, a brute-force approach checks every possible permutation of cities. The number of permutations of n cities is $n!$.

* B: If elements are inserted into a BST in sorted order (e.g., 1, 2, 3, 4, 5), the tree becomes a "straight line" or skewed tree, making it functionally identical to a linked list.

* C: This is a recursive tree where every node branches into two more. At depth n , there are 2^n function calls.

* A: This is the mathematical definition: $g(n)$ is an upper bound if, after a point n_0 , the actual time $T(n)$ is always less than or equal to $g(n)$ multiplied by some constant.

* B: While inserting n elements into a heap one by one is $O(n \log n)$, the Bottom-Up (Floyd's) algorithm takes advantage of the fact that most nodes are near the leaves and don't have far to move, resulting in a mathematical sum that converges to $O(n)$.